



ELEC3609/9609 - Week 9 Lab

08 AWS Cloud & GraphQL

This module introduces key concepts in cloud-based application development using AWS and modern API technologies. You will explore essential AWS services for deployment and security, understand the benefits of using managed databases like Amazon RDS, and learn how GraphQL improves API flexibility and efficiency compared to traditional REST.

Individual Task: Individual Task 8 - Basic Cloud Deployment (AWS Challenge Labs)

Lab Resources: N/A

Learning Objectives

After completing this lab, students should be able to:

- Identify core AWS components for deploying secure and scalable web applications.
- Explain the advantages of using Amazon RDS for collaborative, production-like development.
- Describe how GraphQL works and how it addresses limitations of REST APIs.
- Recognise common security and architectural considerations in cloud and API design.

Chapter 1. AWS Cloud Architecture Overview

AWS provides a flexible and scalable cloud platform with various services to build secure, reliable, and highly available applications. Key components of AWS cloud architecture include compute, storage, networking, security, and management services.

All of you have already been invited to two AWS Cloud courses, check mailbox and accept the invitation.

1.1 Key Components

1. **Amazon EC2 (Elastic Compute Cloud):**

Scalable virtual servers to run applications. Supports various instance types, Auto Scaling, and Load Balancing.

2. **Amazon RDS (Relational Database Service):**

Managed relational database service supporting MySQL, PostgreSQL, Oracle, and others. Offers automated backups, scaling, and multi-AZ deployments.

3. **Amazon VPC (Virtual Private Cloud):**

Isolated network environment for AWS resources with customizable IP ranges, subnets, route tables, and gateways.

4. **Amazon AMI (Amazon Machine Image):**

Pre-configured templates for EC2 instances, including operating system, software, and configurations.

5. **AWS IAM (Identity and Access Management):**

Manages user access and permissions for AWS resources using roles, policies, and multi-factor authentication.

1.2 Security Measures

1. **Internet Gateway:**

Enables internet access for VPC resources.

2. **NAT Gateway:**

Allows private subnets to access the internet securely.

3. **IAM Roles:**

Provides secure access to AWS services for applications without hard-coded credentials.

4. **Parameter Store:**

Secure storage for configuration data and secrets, with encryption using AWS KMS.

5. **Elastic Load Balancer (ELB):**

Distributes incoming traffic for high availability and integrates with WAF for security.

1.3 Additional Services

1. Amazon S3:

Scalable object storage with high durability, versioning, and lifecycle management.

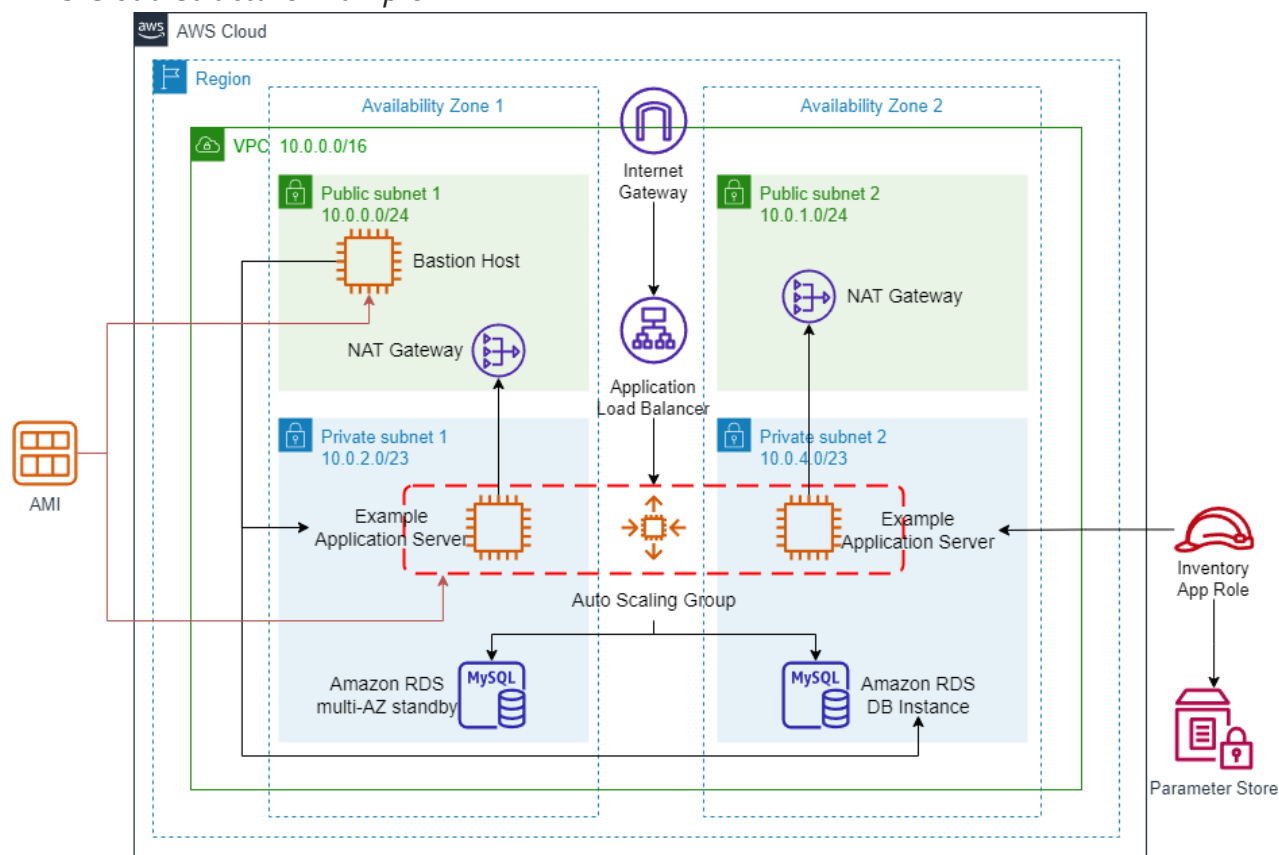
2. AWS CloudFront:

Content Delivery Network (CDN) for low-latency content delivery.

3. AWS CloudWatch:

Monitoring and observability service for metrics, logs, and alerts.

AWS Cloud Structure Example



! INFO

For your final assignment (deployment), you don't need to build a full AWS cloud architecture. Instead, you will generally use an EC2 instance to host both the application and the web server (EC2 + Nginx + uWSGI). You may also consider using Amazon RDS and Amazon S3 as needed.

Chapter 2. Amazon RDS

Amazon Relational Database Service (Amazon RDS) is an easy-to-manage relational database service optimised for total cost of ownership. For more information and lab practices about Amazon RDS, please read:

- [Module 8 – Databases, AWS Academy Cloud Foundations](#)
- [Module 6 – Adding a Database Layer, AWS Academy Cloud Architecting](#)

2.1 Reasons to Use Remote Database Services

1. Centralised and Consistent Database Management:

Using Amazon RDS or any cloud-based database ensures all team members work with a centralised, shared database. This eliminates the need for manually updating local databases when changes are made, reducing errors and ensuring consistency across the team.

2. Avoiding Database Files in Version Control:

Database files like `db.sqlite3` are unsuitable for version control due to their size, frequent changes, and potential security risks. Amazon RDS manages the database remotely, keeping the repository clean and focused on code and migrations rather than raw data.

3. Remote Access and Collaboration:

Amazon RDS allows multiple developers to access the same database remotely, facilitating collaboration in distributed teams. Changes made by one developer are instantly available to others, eliminating the need for sharing database dumps or setting up local databases repeatedly.

4. Simplified Database Migrations:

With Amazon RDS, database migrations are applied to a central instance, providing all developers with immediate access to updated schemas. This streamlines the migration process compared to running scripts on individual local environments, ensuring everyone is on the same page.

5. Development Environment Parity:

Using Amazon RDS in development ensures the environment closely mirrors production, reducing environment-specific issues during deployment. This consistency helps developers catch potential issues early and facilitates smoother production rollouts.

6. Backup, Recovery, and Security:

Amazon RDS offers built-in features for backup, recovery, encryption, and scaling, which are difficult to manage in local environments. This ensures that the development database remains secure and reduces the risk of data loss or unauthorised access.

Chapter 3. GraphQL

GraphQL is a query language for APIs and a runtime for executing those queries against your data. Developed by Facebook in 2012 and released to the public in 2015, GraphQL provides a more efficient, powerful, and flexible alternative to REST for interacting with APIs. Many popular public APIs adopted GraphQL as the default way to access them, including GitHub, Yelp, Shopify.

! INFO

Watch the [following video](#) to get a basic understanding of GraphQL.

ByteByteGo



Watch on

3.1 Limitations of (*Poorly Designed and Managed*) REST

- **Over-fetching and Under-fetching:**

REST APIs often return more data than needed (over-fetching) or too little data (under-fetching), requiring multiple requests to different endpoints.

- **Multiple Endpoints:**

REST typically uses multiple endpoints for different resources (`/users`, `/posts`, `/comments`, etc.). Managing and maintaining these endpoints can be cumbersome as the API evolves or scales.

- **Lack of Flexibility:**

REST responses are often fixed by the server, meaning clients have little control over what data is returned. Changing data needs may require additional endpoints or modifications to the API.

- **Versioning Issues:**

REST APIs commonly use versioning (e.g., `/v1/users`, `/v2/users`) to manage changes, which can lead to bloated and complex API management.

3.2 How GraphQL Works

- **GraphQL Schema:**

The schema is the core of any GraphQL server. It defines the types, queries, and mutations that clients can use to interact with the API.

- **Defining Types in GraphQL:**

Types in GraphQL are the building blocks of a schema. They define the shape of the data that can be queried.

- **Queries:**

Used to fetch data from a GraphQL server. They define what data the client can request (read operations in CRUD).

- **Mutations:**

Used to modify data on the server (create, update, delete operations).

- **Resolvers:**

Functions that provide the logic for fetching or modifying data for each field in a type.

- **Subscriptions:**

Enable real-time capabilities in a GraphQL server, allowing clients to subscribe to certain events.

- **Running a GraphQL Server:**

Can be done using libraries such as Apollo Server, Graphene (Python), etc.

3.3 GraphQL vs REST: Similarities and Drawbacks

Similarities

Aspect	Shared Feature	Notes
Architectural Principles	Stateless, client-server architecture, CRUD operations	Both follow the same high-level principles.
Transport Protocol	Use HTTP as the transport layer	Standard communication method for both.
Data Transfer Formats	JSON (common), REST also supports XML	JSON is the default, but REST allows additional formats.

Drawbacks of GraphQL

Drawback	Explanation
Learning Curve & Implementation Complexity	Requires schemas, types, and resolvers, making setup more complex than REST.
Over-fetching / N+1 Problem	Deeply nested queries can cause N+1 issues if not optimised properly.
Performance with Large Queries	Large, dynamic queries can degrade server performance.
Caching Complexity	Lacks standard HTTP caching; requires custom strategies or tools like Apollo.
Security / Malicious Queries	Flexible queries can be abused (e.g., DoS); requires rate limiting and complexity analysis.
Lack of Built-in File Upload Support	No native file upload support; needs workarounds such as <code>multipart/form-data</code> or <code>graphql-upload</code> .

Further Readings

- [AWS Academy Cloud Foundations](#)
- [AWS Academy Cloud Architecting](#)

Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.